

Exercícios do Dia 2, resolvidos e explicados

Uma solução possível pra cada exercício — não a única, mas uma que passa em todos os requisitos.

COMO USAR ISTO

A ideia não é copiar. Uma forma de estudar que funciona bem: leia a solução de um exercício uma vez, entenda por que ela funciona, feche este PDF e tente escrever a sua versão do zero, sem olhar. Só volte aqui depois, pra comparar. Se o seu código chegou num resultado diferente mas ainda cumpriu os requisitos mínimos do enunciado, está igualmente certo — não existe um único jeito certo de resolver a maioria desses exercícios.

PARTE 1

Criando classes e chamando métodos

Exercício 1

PERSONAGEM COMPLETO

Enunciado: classe *Personagem* com `__init__` e um método `mostrar_status()` que classifica a vida em faixas, usando `if/elif/else`.

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def mostrar_status(self):
        if self.vida >= 70:
            situacao = "saudavel"
        elif self.vida >= 30:
            situacao = "ferido"
        elif self.vida > 0:
            situacao = "critico"
        else:
            situacao = "morto"
        print(self.nome, "-", self.vida, "de vida -", situacao)

thoric = Personagem("Thoric", 100, 15)
elara = Personagem("Elara", 60, 25)

thoric.mostrar_status()
elara.mostrar_status()
```

POR QUE FUNCIONA

As três faixas do `if/elif/else` são checadas em ordem, de cima pra baixo — a primeira condição verdadeira "ganha" e as outras nem são olhadas. Por isso `elif self.vida > 0` não precisa repetir `self.vida < 30`: se o código chegou até ali, já se sabe que não é `>= 70` nem `>= 30`.

Exercício 2

MÉTODO QUE MUDA O ESTADO

Enunciado: *método `receber_dano(quantidade)` que subtrai da vida, sem deixar ficar negativa. Aplicado duas vezes.*

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def mostrar_status(self):
        print(self.nome, "-", self.vida, "de vida")

    def receber_dano(self, quantidade):
        self.vida = self.vida - quantidade
        if self.vida < 0:
            self.vida = 0

thoric = Personagem("Thoric", 100, 15)
thoric.receber_dano(50)
thoric.mostrar_status()      # Thoric - 50 de vida

thoric.receber_dano(60)
thoric.mostrar_status()      # Thoric - 0 de vida (nao -10)
```

POR QUE FUNCIONA

O segundo `receber_dano(60)` deixaria a vida em -10 se nada fosse feito. O `if self.vida < 0: self.vida = 0` roda *depois* da subtração, corrigindo o valor antes que qualquer outro código chegue a ler `self.vida` — por isso o "clamp" (travar num mínimo) precisa vir logo em seguida da conta, e não em outro lugar.

Exercício 3

CLASSE NOVA + FOR DENTRO DO MÉTODO

Enunciado: *classe `Inimigo` com método `atacar_varias_vezes(vezes)` usando `for` por dentro; lista de 3 inimigos percorrida com outro `for`.*

```
class Inimigo:
    def __init__(self, nome, tipo):
        self.nome = nome
        self.tipo = tipo

    def atacar_varias_vezes(self, vezes):
        for i in range(vezes):
            print(self.nome, "ataca!")

inimigos = [
    Inimigo("Goblin", "pequeno"),
    Inimigo("Esqueleto", "morto-vivo"),
    Inimigo("Troll", "grande"),
]

for inimigo in inimigos:
    inimigo.atacar_varias_vezes(2)
```

POR QUE FUNCIONA

Tem dois `for` completamente separados aqui, e é fácil confundir os dois. O `for` de **dentro** do método (`for i in range(vezes)`) controla quantas vezes *aquela* inimigo ataca. O `for` de **fora** (`for inimigo in inimigos`) controla *quais* inimigos são chamados. Um decide "quantas vezes", o outro decide "quem".

Enunciado: `calcular_dano_total(tipo_ataque)` devolvendo (com `return`) o ataque multiplicado conforme o tipo, usando `if/elif/else`.

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque

    def calcular_dano_total(self, tipo_ataque):
        if tipo_ataque == "critico":
            return self.ataque * 2
        elif tipo_ataque == "fraco":
            return self.ataque * 0.5
        else:
            return self.ataque

thoric = Personagem("Thoric", 100, 15)
dano = thoric.calcular_dano_total("critico")
print(dano)          # 30
```

POR QUE FUNCIONA

Cada ramo do `if/elif/else` tem o seu próprio `return` — assim que um deles roda, o método termina ali, sem chegar nos outros. Guardar o resultado em `dano` antes de imprimir (em vez de `print(thoric.calcular_dano_total("critico"))` direto) é o que diferencia esse exercício do Exercício 1: aqui o valor existe pra ser *usado* depois, não só mostrado na hora.

PARTE 2

Classes específicas: Guerreiro e Mago

Exercício 5

Enunciado: `Guerreiro.com_atacar_com_espada()` usando `while`: golpeia enquanto tiver energia, e avisa quando cansar.

```
class Guerreiro:
    def __init__(self, nome, energia):
        self.nome = nome
        self.energia = energia

    def atacar_com_espada(self):
        while self.energia > 0:
            print(self.nome, "golpeia com a espada!")
            self.energia -= 1
        print(self.nome, "esta cansado.")

thoric = Guerreiro("Thoric", 3)
thoric.atacar_com_espada()
```

POR QUE FUNCIONA

`self.energia -= 1` tem que estar *dentro* do `while`, senão a condição nunca muda e o laço não para (o mesmo erro de laço infinito visto no Dia 2). O `print(self.nome, "esta cansado.")`, por outro lado, precisa estar *fora* do `while` — no mesmo nível de indentação dele — pra rodar só uma vez, depois que a energia já chegou a zero.

Enunciado: *Mago com conjurar_ate_acabar()* usando *while*: conjura enquanto tiver mana suficiente, e ao final informa o total conjurado.

```
class Mago:
    def __init__(self, nome, mana):
        self.nome = nome
        self.mana = mana

    def conjurar_ate_acabar(self):
        feiticos = 0
        while self.mana >= 10:
            print(self.nome, "conjura um feitico!")
            self.mana -= 10
            feiticos += 1
        print(self.nome, "conjurou", feiticos, "feiticos no total.")

elara = Mago("Elara", 35)
elara.conjurar_ate_acabar()
```

POR QUE FUNCIONA

`feiticos` é uma variável comum, criada dentro do método — não é um atributo (`self.algo`), porque só interessa durante essa chamada específica do método, não precisa ser lembrada depois. Repare também que com `mana = 35`, o laço roda só 3 vezes ($35 \rightarrow 25 \rightarrow 15 \rightarrow 5$), porque 5 não é mais `>= 10` — sobra mana sem uso, e é assim mesmo que deveria funcionar.

Exercício 7 • desafio

Enunciado: *função duelo(guerreiro, mago)*, fora de qualquer classe, comparando energia e mana com *if/elif/else*.

```
def duelo(guerreiro, mago):
    if guerreiro.energia > mago.mana:
        print(guerreiro.nome, "venceria o duelo.")
    elif mago.mana > guerreiro.energia:
        print(mago.nome, "venceria o duelo.")
    else:
        print("Empate!")

guerreiro1 = Guerreiro("Thoric", 8)
mago1 = Mago("Elara", 40)

duelo(guerreiro1, mago1)      # Elara venceria o duelo.
```

POR QUE FUNCIONA

`duelo` não está indentada dentro de nenhuma classe — ela fica solta no arquivo, no mesmo nível de `class Guerreiro` e `class Mago`. Ela recebe dois objetos como parâmetros comuns (nada de `self`, porque não é método de ninguém) e só lê os atributos de cada um (`guerreiro.energia`, `mago.mana`) pra decidir o resultado. É a mesma ideia do guia sobre várias classes num `main.py`: uma função comum consegue operar sobre objetos de classes diferentes, desde que saiba quais atributos ler.

Se a sua solução ficou diferente dessas mas ainda cumpre os requisitos mínimos do enunciado de cada exercício, ela está certa também — o gabarito mostra uma forma de resolver, não a forma.